
OBJECTIF

Objectif :

- Réaliser des fonctions sous PostgreSQL qui répondent à des règles de gestions.
- Tester ces fonctions en les utilisant dans une requête ou en l'appelant directement.

Ressources :

- Vous disposez du diagramme de classe et du modèle physique de la base *TravelNow* sur laquelle vous devez réaliser les fonctions.
- Du diaporama de cours.

BASE UTILISEE : TRAVELNOW ,

1. Lancez la console d'administration du SGBD PostGres :
ou <https://prenom-nom-etu.pedaweb.univ-amu.fr/phppgadmin/>
2. Récupérez le script de création des tables (*script create table V3.sql*), copier/coller dans la fenêtre SQL
3. Lancer le script
4. Vos tables sont "normalement" créées et renseignées dans le schéma de données *tp2_travelnow*, vérifiez-le en utilisant la partie gauche de phppgadmin

Vous pouvez commencer le TP, à vous de jouer !!

TRAVAIL A FAIRE

PARTIE 1 : FONCTIONS SCALAIRES

Objectif : Créer des fonctions qui retournent une **valeur unique**.

Exercice 1 : Calculer l'âge d'un client

Créer une fonction *calculer_age_client(id_client INTEGER)* qui retourne l'âge (en années) d'un client donné.

Indice : Utiliser `EXTRACT(YEAR FROM AGE(CURRENT_DATE, date_naissance))`.

Exercice 2 : Vérifier si un client est majeur

Créer une fonction *est_majeur(id_client INTEGER)* qui retourne TRUE si le client est majeur (18 ans ou plus), FALSE sinon.

Indice : Utiliser la fonction de l'Exercice 1.

Exercice 3 : Calculer le prix total d'une réservation

Créer une fonction *prix_total_reservation(id_reservation INTEGER)* qui retourne le montant total d'une réservation **en appliquant les promotions** associées au voyage.

Règles :

- Si le voyage a une ou plusieurs promotions actives (date du voyage entre date_debut et date_fin des promotions), appliquer le **pourcentage de réduction le plus élevé**.
- Sinon, retourner le montant de base de la réservation.

Indice :

- Attention le prix du voyage est pour une personne !!

Exercice 4 : mettre le montant des réservations

Créer une fonction *maj_resa()* qui modifie le montant total (colonne *montant* de *reservations*) d'une réservation **si celui-ci ne correspond pas** au montant calculé par la fonction *prix_total_reservation(id_reservation INTEGER)*.

Toutes les réservations seront vérifiées et mises à jour si nécessaire !

Règles :

- Si le montant est différent de celui calculé, alors le mettre à jours !

Exercice 5 : Compter le nombre de réservations d'un client

Créer une fonction *nombre_reservations_client(id_client INTEGER)* qui retourne le nombre total de réservations (tous statuts confondus) pour un client.

Exercice 6 : Calculer le montant moyen des réservations confirmées pour un client

Créer une fonction *montant_moyen_reservations_confirmées(id_client INTEGER)* qui retourne le **montant moyen** des réservations **confirmées** pour un client.

Règles :

- Si le client n'a aucune réservation confirmée, retourner 0.
- Arrondir le résultat à 2 décimales.

PARTIE 2 : FONCTIONS RETOURNANT DES JEUX DE RESULTATS (PLUS D'UNE LIGNE)

Objectif : Créer des fonctions qui **retournent plusieurs lignes**

Exercice 1 : Lister les voyages d'un guide

Créer une fonction *voyages_par_guide(id_guide INTEGER)* qui retourne **tous les voyages** (id_voyage, destination, date_depart, statut) gérés par un guide donné.

Indice :

- Utiliser RETURNS SETOF RECORD ou RETURNS TABLE(...).
- Joindre les tables voyages et destinations.

Exercice 2 : Lister les destinations les plus populaires en nombre de réservations

Créer une fonction *destinations_populaires(rang INTEGER DEFAULT 5)* qui retourne les **destinations** (nom, pays, nombre de réservations) **les plus réservées correspondantes au rang passé en paramètre**, cad que si le rang est 3 on affichera les voyages qui sont au troisième rang en terme de nombre de réservation, le top 1 (rang 1) correspond au voyage qui on le plus de réservation !!

Attention : on ne s'occupe pas des avis dans cette fonction

Règles :

- Trier par nombre de réservations **décroissant**.

Indice :

- Compter les réservations par destination avec COUNT(*) et GROUP BY.

PARTIE 3 : FONCTIONS AVANCEES

Objectif : Utiliser des **curseurs**, des **boucles**, des **exceptions**

Exercice 1 : Lister les réservations non payées

Créer une fonction *reservations_non_payees()* qui retourne toutes les réservations dont le **montant total n'est pas couvert par les paiements** (id_reservation, id_client, montant_reservation, montant_paye, reste à payer).

Attention :

1. **Il ne s'agit que des paiements dans un etat VALIDE et un statut de réservation CONFIRMEE**
2. **il se peut qu'il y ait des cumuls de paiement qui correspondent à des sommes trop versées pour une réservation, cad que le reste à payer est négatif, cela fera l'objet d'une autre question.**
3. **D'autre part il se peut aussi qu'une réservation n'ait pas encore fait l'objet d'un paiement, elle n'apparaît donc pas encore dans la table paiement !!**

Règles :

- Calculer le montant payé avec SUM(montant) dans la table paiements.
- Ne retourner que les réservations où montant_reservation > montant_paye.

Indice :

- Utiliser une jointure gauche (LEFT JOIN) entre reservations et paiements.
- Utiliser un curseur

Exercice 2 : Appliquer une réduction à un voyage

Créer une fonction `appliquer_reduction_voyage(id_voyage INTEGER, pourcentage NUMERIC)` qui :

- **Met à jour** le prix du voyage en appliquant la réduction.
- **Retourne** le nouveau prix.

Règles :

- Vérifier que pourcentage est entre 0 et 100 (sinon, lever une exception).
- Mettre à jour le champ prix de la table voyages.

Indice :

- Utiliser `RAISE EXCEPTION` si le pourcentage est invalide.

Exercice 3 : Annuler une réservation et rembourser

Créer une fonction `annuler_reservation_et_rembourser(id_reservation INTEGER)` qui :

1. **Met à jour** le statut de la réservation en 'ANNULEE'.
2. **Ajoute un paiement** de remboursement pour chaque paiement existant (même montant et `etat = 'REMBOURSE'`).
3. **Retourne** `TRUE` si l'opération a réussi, `FALSE` sinon.

Règles :

- Vérifier que la réservation existe et n'est pas déjà annulée.

Indice :

- Utiliser un curseur pour parcourir les paiements de la réservation.
- Insérer de nouveaux paiements par `VIREMENT` avec `etat = 'REMBOURSE'` si l'état du paiement est valide

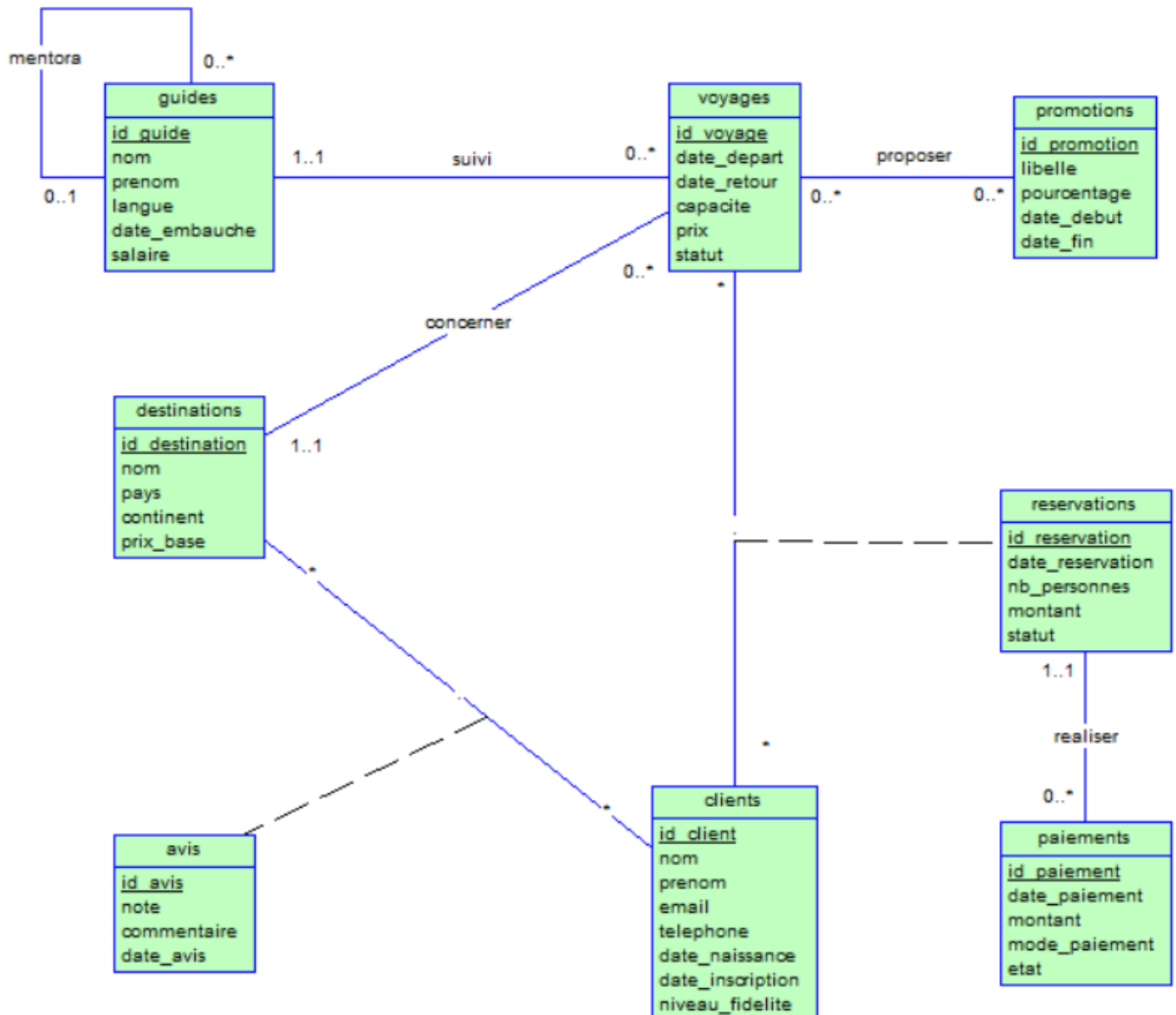
Exercice 4 : Destination avec les meilleurs avis

Créer une fonction `meilleurs_destinations()` qui retourne toutes les informations sur les destinations avec le meilleurs avis.

Indice :

- Calculer la moyenne des notes des avis par des destinations et prendre la plus haute
- Utiliser un curseur pour retourner toutes les informations sur la ou les destinations qui ont la meilleure moyenne.

ANNEXE 1 : DIAGRAMME DE CLASSES DE LA BASE TP2_TRAVELNOW



ANNEXE 2 : MPD DE LA BASE TP2_TRAVELNOW

